MODULE 25 · DOMAIN 4 - TROUBLESHOOTING AND OPTIMIZATION

CloudWatch – Logs, Metrics and Alarms

Metric filters, subscription filters, embedded metrics - and why an alarm on a quiet function never fires.

DOMAIN 4 of 4 EXAM WEIGHT 18% MODULE 25 of 34

admin@sthithapragna.com

Logs are **events**. Metrics are **numbers over time**. Most CloudWatch questions are about moving between the two.

Log groups and log streams

- A **log group** is the container. Retention, encryption and metric filters are set on it.
- A **log stream** is a sequence of events from one source, within a group.
- Lambda creates one group per function, and a stream per execution environment.
- So one stream contains many invocations, and one invocation never spans streams.
- Every event has a timestamp and a message. That is the whole model.

> ON THE EXAM
>
> The Lambda mapping is worth internalising: a stream per execution environment means a cold start creates a new stream, which is why streams appear in bursts after a deployment.

Retention

- **Log groups default to Never Expire.** Logs accumulate and are billed forever.
- Retention is set per group, from **1 day to 10 years**, or never.
- A group created automatically by a service also defaults to never expiring.
- **Export to S3** for long-term archival at a much lower storage cost.
- Changing retention applies to existing events too, so old logs are deleted.

> ON THE EXAM
>
> This is one of the most common sources of unexplained AWS spend. Every Lambda function you ever deployed still has its logs, unless somebody set a retention period.

Currency note: log classes

- **The examinable model:** one kind of log group, priced per GB ingested.
- **Infrequent Access** is a cheaper ingestion class for logs queried only occasionally.
- It supports Logs Insights, export to S3 and encryption, at a lower price per GB.
- It does not support metric filters, subscription filters, Live Tail or alarms.
- **A log group's class cannot be changed after creation.** It is a one-time decision.

> ON THE EXAM
>
> The trade is specific: cheaper ingestion in exchange for losing exactly the features that turn logs into metrics and alerts. Right for forensic logs, wrong for operational ones.

How logs get in

- **AWS services publish directly.** Lambda, API Gateway and others need only permission.
- The **CloudWatch agent** ships system and application logs from EC2 and on-premises servers.
- The **PutLogEvents** API, used directly or through a logging library.
- For Lambda, the execution role needs **logs:CreateLogStream** and **logs:PutLogEvents**.
- A function with no logs at all is usually a permissions problem, not a code problem.

> ON THE EXAM
>
> That last line is a real diagnostic. If a function runs but produces no log output whatsoever, check the execution role before reading any code.

Structured logging

- Log a **JSON object** per event rather than an interpolated string.
- Logs Insights can then filter and aggregate on fields with no **parse** step.
- Include a **correlation ID** on every line, so one request can be traced across functions.
- Lambda's JSON log format adds requestId and log level automatically.
- An interpolated message is a string. Nothing can query inside it reliably.

> ON THE EXAM
>
> The correlation ID is the part people skip and later wish they had. Without it, following one request across five functions means guessing from timestamps.

Lambda logging in detail

- **Log format** can be plain text or **structured JSON**, configured on the function.
- **Application log level** filters your own logs at the platform, before ingestion.
- So debug logging can be left in the code and switched on without redeploying.
- **System log level** controls the platform's own START, END and REPORT lines.
- A custom **log group** can be set, so several functions share one group.

> ON THE EXAM
>
> Filtering at the platform means unwanted log lines are never ingested, so you are not billed for them. Filtering in code still costs you the CPU time.

Metric filters

- A **filter pattern** on a log group matches events and increments a **custom metric**.
- Patterns match terms, phrases, JSON fields, or a regular expression.
- The metric value can be a fixed **1** per match, or a **numeric field** from the event.
- You can then **alarm** on that metric, which you cannot do on logs directly.
- **Metric filters are not retroactive.** They only match events ingested after creation.

> ON THE EXAM
>
> This is the standard answer for 'alarm when this appears in the logs'. The non-retroactive point matters: creating a filter to investigate a past incident finds nothing.

Subscription filters

- Delivers matching log events in **near real time** to another destination.
- Destinations: **Kinesis Data Streams**, **Data Firehose**, **Lambda**, or OpenSearch.
- Firehose to S3 or OpenSearch is the standard centralised-logging pattern.
- **Cross-account** delivery is supported, using a destination in the receiving account.
- Events arrive **base64-encoded and gzip-compressed**, so a Lambda consumer must decompress.

> ON THE EXAM
>
> That encoding detail catches people writing their first log-processing function: the payload is not readable JSON until you have decoded and decompressed it.

Metric filter or subscription filter?

METRIC FILTER

Count it and alarm

- Produces a CloudWatch metric
- Stays inside CloudWatch
- The basis for an alarm
- No destination to run
- Not retroactive

SUBSCRIPTION FILTER

Send it somewhere

- Delivers the events themselves
- Kinesis, Firehose, Lambda, OpenSearch
- For search, archival or analysis
- Cross-account capable
- You run the consumer

THE TIEBREAKER

Ask whether the scenario wants a number or the events. 'Alert when errors exceed 10' is a metric filter; 'centralise logs for searching' is a subscription filter.

CloudWatch Logs Insights

- An interactive query engine over log groups. No indexing to configure.
- Core commands: **fields**, **filter**, **stats**, **sort**, **limit**, **parse**.
- **parse** extracts fields from unstructured text. Structured JSON logs need no parsing.
- Results can be added to a dashboard, and saved for reuse.
- Billed by **data scanned**, so a narrow time range costs less than a wide one.

> ON THE EXAM
>
> The billing model shapes usage: always narrow the time range first. Querying a week when you needed an hour costs roughly 168 times as much.

Queries worth knowing

- **filter @message like /ERROR/** narrows to matching events.
- **stats count() by bin(5m)** produces a time series from log events.
- **stats avg(duration), pct(duration, 99) by operation** compares latency across operations.
- **sort @timestamp desc | limit 20** is the fastest way to see what just happened.
- **@message**, **@timestamp** and **@logStream** are always available as fields.

> ON THE EXAM
>
> stats with bin is the one to remember: it turns log events into a chart, which is often all an investigation needs before deciding where to look next.

Currency note: three query languages

- **The examinable language** is Logs Insights QL, with its pipe-separated commands.
- Logs Insights now also accepts **OpenSearch PPL** and **OpenSearch SQL**.
- SQL brings **JOINs across log groups** and subqueries, which QL cannot do.
- PPL is pipe-based and will feel familiar if you know Logs Insights QL.
- Concurrency differs: 100 concurrent QL queries per account, but only 15 for SQL or PPL.

> ON THE EXAM
>
> Answer Logs Insights QL on the exam. The addition is genuinely useful in practice, because joining two log groups was previously impossible.

Live Tail

- Streams log events into the console **as they are ingested**, with a filter pattern.
- Useful during a deployment or while reproducing a bug deliberately.
- Several log groups can be tailed at once.
- Billed **per minute of tailing**, so it is not something to leave running.
- Standard log class only. Not available on Infrequent Access groups.

> ON THE EXAM
>
> Live Tail fills the gap between an alarm firing and a Logs Insights query: it shows what is happening now, rather than what happened.

Metrics

- A metric is identified by a **namespace**, a **name** and a set of **dimensions**.
- **Each unique dimension combination is a separate metric**, and is billed as one.
- AWS namespaces look like **AWS/Lambda**. Custom ones are anything else.
- Dimensions are how you slice: by function name, by alias, by executed version.
- High-cardinality dimensions such as user ID create an unbounded number of metrics.

> ON THE EXAM
>
> That last point is the classic custom-metric cost mistake. A dimension with a million possible values produces a million billable metrics.

Resolution and statistics

- **Standard resolution** is one-minute granularity. **High resolution** is one second.
- High resolution alarms can evaluate at 10 or 30 seconds, at higher cost.
- Statistics: **Sum**, **Average**, **Minimum**, **Maximum**, **SampleCount** and percentiles.
- **Average hides outliers.** A p99 alarm catches what an average alarm misses.
- Metrics are retained 15 months, aggregated to coarser periods as they age.

> ON THE EXAM
>
> Alarming on average latency is a common mistake. If 1% of requests take 30 seconds, the average barely moves and the p99 tells you immediately.

Custom metrics

- **PutMetricData** publishes a value into a namespace you choose.
- It is a **synchronous API call**, so it adds latency to whatever path calls it.
- It is billed per call, and per metric stored.
- Values can be sent as a **statistic set**, pre-aggregated, to reduce call volume.
- In a Lambda function, a failed PutMetricData call can fail the invocation.

> ON THE EXAM
>
> The synchronous cost is the reason the next slide exists. Adding a metric call to a hot path means every request pays for the observability.

Embedded Metric Format

- Write a specially structured **JSON log line**, and CloudWatch extracts metrics from it.
- No synchronous API call, so no added latency and no failure path.
- The log line keeps **full context** alongside the metric values.
- So you can alarm on the metric and then read the exact events behind it.
- High-cardinality fields can be logged as properties without becoming dimensions.

> READ THIS TWICE
>
> That last point is the real advantage. Log the user ID as a property so it is queryable, without creating a billable metric per user.

PutMetricData or EMF?

PUTMETRICDATA

Direct API call

- Immediate, explicit publication
- Works from anywhere with credentials
- Adds latency to the calling path
- Billed per API call
- No surrounding context

EMBEDDED METRIC FORMAT

A structured log line

- Asynchronous, via the log pipeline
- No latency and no failure path
- Context preserved with the metric
- Billed as log ingestion
- The recommended default in Lambda

THE TIEBREAKER

For anything running in Lambda, EMF is almost always right. PutMetricData suits a process publishing occasional aggregate values rather than per-request data.

Alarms

- A metric, a **statistic**, a **period**, a **threshold** and a comparison operator.
- **Evaluation periods** and **datapoints to alarm** give you M-of-N behaviour.
- So 3 of 5 datapoints breaching avoids alerting on a single spike.
- States are **OK**, **ALARM** and **INSUFFICIENT_DATA**.
- Actions can fire on entering any state, not only ALARM.

> ON THE EXAM
>
> M-of-N is the fix for a flapping alarm. Widening the period smooths the data; requiring 3 of 5 datapoints tolerates a spike without missing a sustained problem.

Missing data

- **missing** is the default. The datapoint is ignored and the state does not change.
- **notBreaching** treats missing data as good, so the alarm goes OK.
- **breaching** treats missing data as bad, so the alarm fires.
- **ignore** keeps the current state indefinitely.
- A metric with **no data at all** means no alarm. Errors publish nothing when nothing runs.

> ON THE EXAM
>
> This is why an error alarm on a low-traffic function can stay silent through an outage: with no invocations there are no datapoints, so there is nothing to breach.

Anomaly detection

- CloudWatch learns a metric's normal **band** from up to two weeks of history.
- The alarm fires when a value falls outside that band, not outside a fixed number.
- It accounts for **daily and weekly seasonality**, so quiet nights are expected.
- Useful where the right threshold genuinely changes over time.
- It needs history to learn from, so it is useless on a brand-new metric.

> ON THE EXAM
>
> The case for it is traffic that varies by time of day: a fixed threshold is either too high at 3am or too low at midday, and anomaly detection is neither.

Composite alarms

- An alarm whose state is a **boolean expression** over other alarms.
- Alert only when errors are high **and** latency is high, rather than on either.
- Suppress child alarms during a known maintenance window with a suppressor alarm.
- One notification for a correlated failure, instead of twenty.
- Composite alarms cannot invoke every action type. Check what the destination supports.

> ON THE EXAM
>
> The suppressor is the underused feature. It stops a single root cause producing a page from every downstream alarm that noticed it.

Alarm actions

- **SNS notification**, which is how alarms reach people or other systems.
- **Auto Scaling** actions, and EC2 actions such as stop, terminate, reboot or recover.
- **Systems Manager** actions, including creating an OpsItem or an incident.
- Alarms also emit **EventBridge events**, which reach anything at all.
- This is the same alarm mechanism CodeDeploy, AppConfig and CloudFormation use for rollback.

> ON THE EXAM
>
> Worth noticing how much of this course routes through one CloudWatch alarm: deployment rollback, configuration rollback and scaling all read the same signal.

CloudWatch Synthetics

- A **canary** is a scripted Lambda function that runs on a schedule against your endpoint.
- It checks availability and latency the way a user would, from outside the system.
- Blueprints cover heartbeat checks, API checks and broken-link checks.
- Failures produce metrics you can alarm on, plus screenshots and a HAR file.
- Distinct from a canary **deployment**. The same word, two unrelated meanings.

ON THE EXAM

Synthetics catches the outage your internal metrics miss: DNS, certificates and the load balancer are all outside the code that publishes your metrics.

Dashboards and Metrics Insights

- Dashboards are global, and can display metrics from **several Regions and accounts**.
- **Metric math** computes derived values, such as an error rate from two metrics.
- **Metrics Insights** is a SQL-like query language over metrics, evaluated at query time.
- It can rank and aggregate across dimensions without listing every metric.
- Logs Insights query results can be added to a dashboard alongside metric widgets.

> ON THE EXAM
>
> Metric math is worth knowing by name: error count and invocation count are separate metrics, and an error rate alarm needs the ratio of the two.

Controlling CloudWatch cost

- **Log ingestion** is usually the largest line. Filter at the source, not after ingestion.
- **Retention set to never expire** is the second. Set a period on every log group.
- **Custom metrics** are billed per metric, and dimensions multiply the count.
- Lambda's application log level filters before ingestion, so unwanted lines cost nothing.
- Infrequent Access reduces ingestion cost where the features are not needed.

> ON THE EXAM
>
> The order matters: ingestion, then retention, then custom metrics. Debug logging left on in production is usually the single biggest contributor.

Ask first: does the scenario want **a number**, or does it want **the events**?

> An application writes a log line containing PaymentDeclined whenever a card is refused. The operations team wants an alert when more than 50 declines occur in five minutes, and wants to see the trend on a dashboard.
>
> The application code cannot be modified, and the log group is in the Standard class.

How should this be implemented?

A. A subscription filter to a Lambda function that counts occurrences and sends an SNS message

B. A metric filter on the log group matching PaymentDeclined, with a CloudWatch alarm on the resulting metric

C. A scheduled Logs Insights query that counts matching lines and publishes a metric

D. Modify the application to call PutMetricData when a decline occurs

◆ PAUSE AND DECIDE

How should this be implemented?

A. Subscription filter plus Lambda

Rebuilds a metric filter by hand, including the counting window and the alerting, with a function to run and pay for.

B. Metric filter plus alarm

Exactly what metric filters exist for. The pattern produces a metric, the alarm watches it, and the metric charts on a dashboard for free.

C. Scheduled Logs Insights query

Logs Insights is interactive and billed by data scanned. Running it on a schedule is expensive and adds delay before the alert.

D. Modify the application

The scenario states the code cannot be modified. It would also add a synchronous API call to the payment path.

THE SIGNAL

Three requirements met by one feature: match a pattern, produce a metric, alarm on it. The 'cannot modify code' clause rules out the only alternative that would be reasonable.

> A company runs 200 Lambda functions across 12 accounts. A central security team needs all log events searchable in one Amazon OpenSearch domain within a minute of being written, for correlation and investigation.
>
> They do not need alarms on the logs themselves, and the volume is high.

What should be configured?

A. A metric filter on each log group, publishing to a shared namespace

B. A nightly export task from each log group to S3, loaded into OpenSearch

C. Subscription filters on the log groups, delivering to Data Firehose and on to OpenSearch

D. Move all log groups to the Infrequent Access class and query them with OpenSearch SQL

◆ PAUSE AND DECIDE

What should be configured?

A. Metric filters

Produces counts, not events. The security team needs to read and correlate the log lines themselves, which a metric cannot give them.

B. Nightly export

The requirement is within a minute. A nightly job is off by three orders of magnitude.

C. Subscription filters to Firehose

Near real-time delivery of the events themselves, supports cross-account destinations, and Firehose handles buffering and delivery at high volume.

D. Infrequent Access plus SQL

Leaves logs in each account rather than centralising them, and IA does not support subscription filters at all.

THE SIGNAL

'Searchable' plus 'within a minute' means the events, delivered in near real time. That is the subscription filter definition almost word for word.

> A Lambda function processing orders should record order value, processing duration and the customer tier for each order. The team wants to alarm on average order value by tier, and to investigate individual slow orders by customer ID afterwards.
>
> Latency is critical and there are around 2,000 invocations per second.

How should the instrumentation be done?

A Emit Embedded Metric Format log lines, with tier as a dimension and customer ID as a property

B Call PutMetricData once per invocation, with customer ID and tier as dimensions

C Write plain log lines and create metric filters for each customer tier

D Call PutMetricData asynchronously in a background thread after returning the response

◆ PAUSE AND DECIDE

How should the instrumentation be done?

A EMF, tier as dimension, customer as property

No API call so no added latency, tier is a low-cardinality dimension you can alarm on, and customer ID stays queryable in Logs Insights without becoming a metric.

B PutMetricData with customer as a dimension

Two problems: a synchronous call on every invocation at 2,000 per second, and customer ID as a dimension creates a billable metric per customer.

C Metric filters per tier

Workable for counts, and it means a filter per tier to maintain, and extracting numeric values like order value is awkward.

D Background thread

Lambda freezes the execution environment after the handler returns, so background work may never run. Module 3 covered this.

THE SIGNAL

The dimension-versus-property distinction is the whole question. Tier has a handful of values and belongs as a dimension; customer ID has millions and belongs as a property.

> An alarm watches the Errors metric of a Lambda function that runs a few times an hour. During a recent incident the function failed on every invocation for three hours, and the alarm never left the INSUFFICIENT_DATA state.
>
> The alarm uses a one-minute period with one datapoint to alarm, and default missing data treatment.

What is the correct fix?

A. Set missing data treatment to breaching so the alarm fires when no data arrives

B. Switch the metric to high resolution so datapoints arrive every second

C. Add a composite alarm combining Errors with Duration

D. Widen the period to match the invocation rate, and alarm on an error rate using metric math

◆ PAUSE AND DECIDE

What is the correct fix?

A Missing data as breaching

The tempting one. It would fire during the incident and also fire every quiet minute, because a function invoked a few times an hour is missing data most of the time.

B High resolution

Publishes datapoints more often only when there is data. A function that is not invoked still produces nothing.

C Composite alarm

Combines alarms that already work. Both underlying metrics have the same gap, so combining them changes nothing.

D Widen the period, alarm on error rate

A period long enough to contain invocations produces datapoints, and an error rate from Errors divided by Invocations is meaningful at any volume.

THE SIGNAL

The mismatch is between a one-minute period and an hourly invocation rate. Option A treats the symptom and would page the on-call engineer all night.

What to carry forward

- **Metric filter for a number, subscription filter for the events.** That splits most questions.
- **EMF over PutMetricData in Lambda:** no latency, no failure path, context preserved.
- **Dimensions are billable and multiply.** High-cardinality values belong as properties.
- Missing data treatment explains an alarm that never fires on a quiet resource.
- Log groups never expire by default, which is the commonest source of surprise cost.

> ON THE EXAM
>
> Next: tracing with X-Ray and Application Signals, which answers the question metrics and logs cannot - where in the call chain the time actually went.